

Add conditional behavior in Kotlin

About this codelab

1. Before you begin

In this Kotlin codelab you will create another dice game, Lucky Dice Roll, trying to roll a lucky number. Your program will set a lucky number and roll the dice. You then check the roll against the lucky number and print an appropriate message to the output. To accomplish this, you will learn how to compare values and make different decisions in your Kotlin program.

To help you focus on the programming concepts without having to worry about the user interface of the app, you will use the browser-based Kotlin programming tool and output your results to the console.

Prerequisites

- How to open, edit, and run code in <https://developer.android.com/training/kotlinplayground>
- Ability to create and run a Kotlin program that uses variables, functions with arguments, classes with methods, and prints a result to the console.

What you'll learn

- How to use `if` and `else` statements.
- How to compare values using operators such as greater than (`>`), less than (`<`), and equal to (`==`).
- How to use `when` statements to choose an option based on a given value.
- What the `Boolean` data type is and how to use its `true` and `false` values for decision making.

What you'll build

- A Kotlin-based dice game, Lucky Dice Roll, which allows you to specify a lucky number. A player will win if they roll the lucky number.

What you need

- A computer with an internet connection.

2. Decision making within your code

Your Lucky Dice Roller program needs to determine whether the user has rolled a lucky number and gets a congratulations, or instead, gets a message to try again.

As an app developer, you'll need to make decisions about how the app should behave and create different outcomes for your users.

If you are building a shopping app, you may show different screens based on the delivery options a user chooses. For a quiz game, you would show different screens based on whether the player's answer is correct. Depending on the app, there could be many possible outcomes, which you'll want to account for in your code.

In your Lucky Dice Roller program, the app should handle different cases such as:

- **If** the dice roll is the lucky number, then display a congratulations message!
- **Else if** the dice roll is not the lucky number, then display a message to try again.

To add this logic to your code, use special Kotlin keywords such as `if` and `else` and `when`.

Let's look at some examples.

Use an `if` statement to set up a condition that is met

1. Inspect the code below. Can you guess what the output is going to be?

```
fun main() {  
    val num = 5  
    if (num > 4) {  
        println("The variable is greater than 4")  
    }  
}
```

2. Copy and paste the code in the Kotlin program editor and run the program to observe the output.

The variable is greater than 4

The decision-making process for this program is:

1. Create a variable `num` and set it to 5.
2. If it is true that `num` is greater than 4, print "The variable is greater than 4".
3. In all other situations, do nothing.

In the above example, `num` is set to 5. The `if` statement compares if the variable is greater than 4. Since that is true, the system next executes the instructions in the curly braces and prints the message.

Recap: The `>` symbol is an operator for comparing two values, whether the first value is greater than the second value, and returning a result of `true` or `false`. Other common operators are: `<` for less than, `==` for equal, `>=` for greater or equal, and `<=` for less than or equal.

Notice the general format for the `if` statement:

- Start with the `if` keyword.
- Follow with two parentheses `()`. Inside the parentheses goes the condition. The condition is anything that can be `true` or `false`. For example, whether a number is greater than another number.
- Follow with two curly braces `{}`. Inside the curly braces you put the code to execute if the condition is `true`.

```
if (condition-is-true) {  
    execute-this-code  
}
```

Boolean data type for true and false values:

- For a program to test whether a condition is met is called "evaluating the condition". The result of evaluating a condition is `true` if it is met, or `false` if it is not met. Developers also say, "the condition evaluates to `true`" or "the condition evaluates to `false`".
- Conditions can only evaluate to `true` or `false`.
- Just like there is a data type `Int` for integers, and `IntRange` for ranges, there is a data type for `true` and `false`, called `Boolean`. You will encounter `Boolean` type variables later in this course.

Use an `if` statement to set up a condition that is not met

1. Change the value of `num` to 3, as shown below. What do you expect to happen if you run this code?

```
fun main() {  
    val num = 3  
    if (num > 4) {  
        println("The variable is greater than 4")  
    }  
}
```

2. Copy and paste the code in the Kotlin program editor and run the program to observe the empty output.

With `num` set to 3, nothing is printed, because the value of `num` is less than 4. So the condition that `num` be greater than 4 is `false`, and the code between the curly braces does not execute, and nothing is printed.

Use `else` to create an alternative for failed conditions

Rather than doing nothing, you can offer your users an alternative when a condition is not met. Just like with regular language, you can do that with an `else` statement.

1. Add an `else` statement to print a message for when `num` is not greater than 4, as shown below. What do you expect to happen if you run this code?

```
fun main() {  
    val num = 3  
    if (num > 4) {  
        println("The variable is greater than 4")  
    } else {  
        println("The variable is less than 4")  
    }  
}
```

2. Copy and paste the code in the Kotlin program editor and run the program to observe the output.

The variable is less than 4

3. Notice that when `num` has a value of 3, the program prints the message "The variable is less than 4" associated with the `else` statement because `num` is not greater than 4.
4. Change `num` to 5 and run the program again. Now it is true that `num` is greater than 4 and the program prints "The variable is greater than 4".
5. Change `num` to 4 and run your program. Now 4 is not greater than 4, and the program prints "The variable is less than 4".

While "The variable is less than 4" is the correct output for the conditions you set in the code, that printed statement is not accurate, since 4 is not less than 4. What you can do is add another condition that checks for the third possibility, whether `num` is exactly 4, and prints a correct statement when that condition is true.

Use an `else if` combination to add alternative conditions

You can have more than one condition. For your example, this is how you can cover all possibilities for `num`:

- **If** `num` is greater than 4, print "The variable is greater than 4".
- **Else if** `num` is equal to 4, print "The variable is equal to 4".
- **Else**, print "The variable is less than 4".

These are referred to as different cases in the if-else statement. There are 3 cases listed.

The updated code looks as follows:

```
fun main() {  
    val num = 4  
    if (num > 4) {
```

```

        println("The variable is greater than 4")
    } else if (num == 4) {
        println("The variable is equal to 4")
    } else {
        println("The variable is less than 4")
    }
}

```

Notice the following changes:

- The value of `num` is now set to 4 so you can test the new condition.
 - Between the original `if` and `else` statements is a new `else if` statement for the case when `num` is exactly 4.
1. Copy and paste the above code in the Kotlin program editor and run the program to observe the output.

The variable is equal to 4

2. Experiment with changing the value of `num`, and seeing how it affects the output. Change `num` to 2 and 6, so that you can see each of the conditions come `true`.

Control flow

When you look at the above if-else statements, the code will execute, or flow, as controlled by the conditions. So the way you guide execution with these conditionals is called the "control flow" of the program.

- Let's say your dice roll `num` is 3. The program checks the first condition (`num > 4`). This is false, so the program checks the next condition (`num == 4`) which is also false. Then the program executes the code of the `else` statement, which is the final option.
- If your dice roll `num` is 6, the first condition (`num > 4`) is true. The program prints the "The variable is greater than 4" message. Since this condition is true, it doesn't have to check the rest, and is done with the if-else statement.
- Use an `else + if` combination to add alternative conditions.

3. Create the Lucky Dice Roll game

In this section, using what you learned in the previous task, you will update the Dice Roller program to check whether you have rolled a preset lucky number. If you have, you win!

Set up your starter code

You are starting the Lucky Dice Roller with code that is similar to the solution code of the previous Kotlin Dice Roller program. You can edit the `main()` function in your previous code to match, or you can copy and paste the code below to get started.

```

fun main() {
    val myFirstDice = Dice(6)
    val rollResult = myFirstDice.roll()
    println("Your ${myFirstDice.numSides} sided dice rolled ${rollResult}!")
}

class Dice (val numSides: Int) {

    fun roll(): Int {
        return (1..numSides).random()
    }
}

```

Note: Everytime you run the above program in the playground, the `main()` function is called. This creates a new instance of `Dice` and calling `roll()` method on it. So everytime you run the program, the value of the dice roll may be different.

Check if the lucky number has been rolled

Create a lucky number first, and then compare the dice roll with that number.

1. In `main()`, delete the `println()` statement.
2. In `main()`, add a `val` called `luckyNumber` and set it to 4. Your code should look like this.

```

fun main() {
    val myFirstDice = Dice(6)
    val rollResult = myFirstDice.roll()
    val luckyNumber = 4
}

```

3. Below, add an `if` statement with a condition inside the parentheses `()` that checks if `rollResult` is equal (`==`) to `luckyNumber`. Leave some room between the curly braces `{}` so you can add more code.

```

fun main() {
    val myFirstDice = Dice(6)
    val rollResult = myFirstDice.roll()
    val luckyNumber = 4
    if (rollResult == luckyNumber) {

    }
}

```

4. Inside the curly braces `{}`, add a `println` statement to print "You win!"

```

fun main() {
    val myFirstDice = Dice(6)
    val rollResult = myFirstDice.roll()
    val luckyNumber = 4

    if (rollResult == luckyNumber) {

```

```
        println("You win!")
    }
}
```

5. Run your program. You may need to run it several times before you get lucky and see the winning message in the output!

You win!

Respond when the lucky number has not been rolled

Getting no feedback from the program if the user didn't win can leave them wondering if the program is broken. It is a good practice to always provide a response when the user does something. For the Lucky Dice Roller program, you can let them know they didn't win by using an `else` statement.

1. Add an `else` statement to print "You didn't win, try again!".

```
fun main() {
    val myFirstDice = Dice(6)
    val rollResult = myFirstDice.roll()
    val luckyNumber = 4

    if (rollResult == luckyNumber) {
        println("You win!")
    } else {
        println("You didn't win, try again!")
    }
}
```

2. Run the program, and no matter the outcome, your users are always notified.

At this point, users know if they won or not, but not why. Always give users information so they understand the result of their actions! Imagine your program was a loan application. "You didn't get approved because your credit rating is poor," is a lot more informative than, "Sorry, no loan for you, try again!" For Lucky Dice Roller, you can give users a different informative message for each roll if they lost. Use multiple `else if` statements to accomplish this.

3. Add `else if` statements to print a different message for each roll. Refer to the format you learned in the previous task, if necessary.

```
fun main() {
    val myFirstDice = Dice(6)
    val rollResult = myFirstDice.roll()
    val luckyNumber = 4

    if (rollResult == luckyNumber) {
        println("You win!")
    } else if (rollResult == 1) {
        println("So sorry! You rolled a 1. Try again!")
    }
}
```

```

    } else if (rollResult == 2) {
        println("Sadly, you rolled a 2. Try again!")
    } else if (rollResult == 3) {
        println("Unfortunately, you rolled a 3. Try again!")
    } else if (rollResult == 5) {
        println("Don't cry! You rolled a 5. Try again!")
    } else {
        println("Apologies! You rolled a 6. Try again!")
    }
}

```

In the code above, you

- Check whether the `rollResult` is the `luckyNumber`.
- If the `rollResult` is the `luckyNumber`, print the winning message.
- Otherwise, check whether the `rollResult` is 1, and if so, print a try again message.
- Otherwise, check whether the `rollResult` is 2, and if so, print a different try again message.
- Otherwise, keep checking through the number 5.
- If the number hasn't been any of 1 - 5, the only option left is 6, so there is no need for another test with `else if`, and you can just catch that last option with the final `else` statement.

Tip: You can have only one `if` statement with one `else` statement in an if-else code block, but in between, you can have as many `else if` statements as you need.

Because having multiple `else if` cases is very common, Kotlin has a simpler way of writing them.

4. Use a when statement

Testing for many different outcomes, or cases, is very common in programming. Sometimes, the list of possible outcomes can be very long. For example, if you were rolling a 12-sided dice, you'd have 11 `else if` statements between the success and the final `else`. To make these kinds of statements easier to write and read, which helps avoid errors, Kotlin makes available a `when` statement.

You are going to change your program to use a `when` statement. A `when` statement starts with the keyword `when`, followed by parentheses `()`. Inside the parentheses goes the value to test. This is followed by curly braces `{}` for the code to execute for different conditions.

1. In your program, in `main()`, select the code from the first `if` statement to the curly brace `}` that closes the last `else` statement and delete it.

```

fun main() {
    val myFirstDice = Dice(6)
    val rollResult = myFirstDice.roll()

```

```

    val luckyNumber = 4
}

```

2. In `main()`, below the declaration of `luckyNumber`, create a `when` statement. Because your `when` needs to test against the rolled result, put `rollResult` in between the parentheses `()`. Add curly braces `{}` with some extra spacing, as shown below.

```

fun main() {
    val myFirstDice = Dice(6)
    val rollResult = myFirstDice.roll()
    val luckyNumber = 4

    when (rollResult) {

    }
}

```

As before, first test whether `rollResult` is the same as the `luckyNumber`.

1. Inside the curly braces `{}` of the `when` statement, add a statement that tests `rollResult` against `luckyNumber`, and if they are the same, print the winning message. The statement looks like this:

```
luckyNumber -> println("You win!")
```

This means:

- You first put the value you are comparing to `rollResult`. That's `luckyNumber`.
- Follow that with an arrow `(->)`.
- Then add the action to perform if there is a match.

Read this as, "If `rollResult` is `luckyNumber`, then print the "You win!" message."

And your `main()` code looks like this.

```

fun main() {
    val myFirstDice = Dice(6)
    val rollResult = myFirstDice.roll()
    val luckyNumber = 4

    when (rollResult) {
        luckyNumber -> println("You win!")
    }
}

```

2. Use the same pattern to add lines and messages for the possible rolls 1 - 6 except 4, as shown below. Your finished `main()` function should look like this.

```

fun main() {
    val myFirstDice = Dice(6)

```

```

val rollResult = myFirstDice.roll()
val luckyNumber = 4

when (rollResult) {
    luckyNumber -> println("You won!")
    1 -> println("So sorry! You rolled a 1. Try again!")
    2 -> println("Sadly, you rolled a 2. Try again!")
    3 -> println("Unfortunately, you rolled a 3. Try again!")
    5 -> println("Don't cry! You rolled a 5. Try again!")
    6 -> println("Apologies! You rolled a 6. Try again!")
}
}

```

3. Run your program. There is no difference in the output, but your code is much more compact and easier to read.

Congratulations! You have learned two ways of printing messages depending on a condition. This is a powerful tool for writing interesting programs!

5. Solution code

```

fun main() {
    val myFirstDice = Dice(6)
    val rollResult = myFirstDice.roll()
    val luckyNumber = 4

    when (rollResult) {
        luckyNumber -> println("You won!")
        1 -> println("So sorry! You rolled a 1. Try again!")
        2 -> println("Sadly, you rolled a 2. Try again!")
        3 -> println("Unfortunately, you rolled a 3. Try again!")
        5 -> println("Don't cry! You rolled a 5. Try again!")
        6 -> println("Apologies! You rolled a 6. Try again!")
    }
}

class Dice(val numSides: Int) {
    fun roll(): Int {
        return (1..numSides).random()
    }
}

```

6. Summary

- Use an `if` statement to set a condition for executing some instructions. For example, if the user rolls the lucky number, print a winning message.
- The `Boolean` data type has values of `true` and `false` and can be used for decision making.
- Compare values using operators such as greater than (`>`), less than (`<`), and equal to (`==`).
- Use a chain of `else if` statements to set multiple conditions. For example, print a different message for each possible dice roll.

- Use an `else` statement at the end of a chain of conditions to catch any cases that may not be covered explicitly. If you cover the cases for 6-sided dice, an `else` statement would catch the 7 and 8 numbers rolled with an 8-sided dice.
- Use a `when` statement as a compact form of executing code based on comparing a value.

General form of if-else:

```
if (condition-is-true) {
    execute-this-code
} else if (condition-is-true) {
    execute-this-code
} else {
    execute-this-code
}
```

When statement:

```
when (variable) {
    matches-value -> execute-this-code
    matches-value -> execute-this-code
    ...
}
```

7. Learn more

- [Vocabulary for Android Basics in Kotlin](#)
- [Kotlin: Control flow](#)
- [Kotlin: when](#)
- [Conditionals](#)

8. Practice on your own

Note: Practices are optional. They provide an opportunity for you to practice what you've learned in this codelab, and sometimes include a coding challenge.

Do the following:

1. Change `myFirstDice` to have 8 sides and run your code. What happens?

Hint: When you increase the number of sides, your `when` statement does not cover all the cases anymore, so for uncovered cases, nothing is printed.

2. Fix the `when` statement to account for all 8 sides. You can do this by adding cases for the additional numbers. **Challenge:** Instead of adding a new case for each number, use an `else` statement to catch all cases that are not covered explicitly.

Hint: You can add more cases to cover more sides. That is a good way to do this, if you want a different message for each number that can be rolled. Or, you can use an `else` statement and print the same message for all sides greater than the 6 covered by the current code.

3. Change `myFirstDice` to have only 4 sides. What happens?

Hint: Changing the number of sides of the dice to less than what is covered by the `when` statement has no noticeable effect, since all the cases that can occur are covered.